

Practical Lab Guide Sessions 1-5

Hands-On Exercises for Foundational Security Concepts

Networking • Operating Systems • IAM • Cryptography • Infrastructure Design

Introduction: Learning by Doing

Security engineering is a practical discipline. Reading about concepts is essential, but **hands-on practice** cements understanding and builds the muscle memory you'll need in real-world scenarios and technical interviews.

This lab guide provides practical exercises for Sessions 1-5. Each lab includes:

- **Objective:** What you'll learn
- **Prerequisites:** Tools and knowledge needed
- **Step-by-step instructions**
- **Expected output:** What success looks like
- **Exploration challenges:** Deepen your understanding

Lab Environment Setup: Most labs can be completed with free tools and a modest computer. Where possible, we use Docker for easy setup/teardown.

SESSION 1 LABS: Networking & TCP/IP

■ Lab 1.1: Subnet Calculation Practice

Objective: Master subnetting through hands-on calculation and verification

Duration: 30 minutes

Tools: Calculator, subnet calculator for verification (ipcalc, sipcalc, or online tool)

Exercise 1: Given 192.168.10.0/24, create 4 equal subnets

Step 1: Calculate new prefix length

- Need 4 subnets $\rightarrow 2^2 = 4$, so borrow 2 bits
- New prefix: $/24 + 2 = /26$

Step 2: Calculate subnet ranges

- Subnet size: $2^{(32-26)} = 64$ addresses
- Subnet 1: 192.168.10.0/26 (hosts: .1 to .62, broadcast: .63)
- Subnet 2: 192.168.10.64/26 (hosts: .65 to .126, broadcast: .127)
- Subnet 3: 192.168.10.128/26 (hosts: .129 to .190, broadcast: .191)
- Subnet 4: 192.168.10.192/26 (hosts: .193 to .254, broadcast: .255)

Step 3: Verify with tool

```
$ ipcalc 192.168.10.0/26
```

Practice Problems:

1. 10.0.0.0/8 $\rightarrow$ create 256 subnets (answer: /16)
2. 172.16.0.0/16 $\rightarrow$ create subnets for 50 hosts each (answer: /26, supports 62 hosts)
3. What subnet does 192.168.5.37/27 belong to? (answer: 192.168.5.32/27)

■ Lab 1.2: Network Reconnaissance with Command-Line Tools

Objective: Use networking tools to discover hosts, services, and network topology

Duration: 45 minutes

Tools: ping, traceroute, nslookup/dig, nmap, netstat

Part 1: Basic Discovery

```
# Ping sweep to find live hosts (Linux/Mac)
for i in {1..254}; do ping -c 1 -W 1 192.168.1.$i | grep 'bytes from' & done

# Or use nmap for faster results
nmap -sn 192.168.1.0/24

# Trace route to understand network path
traceroute google.com

# Windows: tracert google.com

# DNS lookup
nslookup google.com
dig google.com ANY
```

Part 2: Port Scanning (Use ONLY on your own network or with permission!)

```
# Scan common ports on a host
nmap -F 192.168.1.1

# Service version detection
nmap -sV 192.168.1.1

# OS detection
sudo nmap -O 192.168.1.1

# Check your own listening ports
netstat -tuln # Linux
netstat -an | findstr LISTENING # Windows
```

Analysis Questions:

1. How many hops to google.com from your location?
2. What services are running on your router?
3. Which ports are listening on your machine? Should they be?
4. Can you identify your DNS servers?

■ Lab 1.3: Packet Capture and Analysis with Wireshark

Objective: Capture and analyze network traffic to understand TCP handshake and DNS queries

Duration: 60 minutes

Tools: Wireshark (free download from [wireshark.org](https://www.wireshark.org))

Setup:

1. Install Wireshark
2. Start capture on your main network interface
3. Apply display filter: `tcp.port == 80` or `dns`

Exercise: Observe TCP Three-Way Handshake

1. Start Wireshark capture
2. In terminal: `curl http://example.com`
3. Stop capture
4. Filter: `tcp.stream eq 0`
5. Find the handshake:
 - Packet 1: SYN (flags: S)
 - Packet 2: SYN-ACK (flags: SA)
 - Packet 3: ACK (flags: A)
6. Right-click → Follow → TCP Stream to see full conversation

Analysis Tasks:

1. What is the initial sequence number (ISN)?
2. What is the window size?
3. How many bytes were transferred?
4. Identify the HTTP GET request and response
5. Apply filter `dns` - observe DNS queries for `example.com`

■ **Challenge:** Capture traffic while browsing HTTPS site. Notice encrypted payload—you can see IPs and ports but not content. This demonstrates why HTTPS is critical.

SESSION 2 LABS: Operating Systems & System Hardening

■ Lab 2.1: Linux File Permissions Mastery

Objective: Understand and manipulate Linux permissions and special bits

Duration: 45 minutes

Environment: Linux VM or WSL

```
# Create test environment
mkdir ~/perms_lab && cd ~/perms_lab
touch file1.txt
mkdir dir1

# Basic permissions
ls -l file1.txt
# Output: -rw-rw-r-- 1 user user 0 date file1.txt

# Change permissions
chmod 644 file1.txt # rw-r--r--
chmod 600 file1.txt # rw-----
chmod 755 file1.txt # rwxr-xr-x

# Symbolic method
chmod u+x file1.txt # Add execute for user
chmod g-w file1.txt # Remove write for group
chmod o= file1.txt # Remove all for others

# Change ownership
sudo chown nobody:nogroup file1.txt
```

SUID/SGID Exercise:

```
# Find SUID binaries (potential privilege escalation vectors)
find / -perm -4000 -type f 2>/dev/null

# Create SUID binary (demo only - dangerous in production!)
echo '#!/bin/bash\nwhoami' > /tmp/suid_test
chmod 755 /tmp/suid_test
./suid_test # Shows your username

sudo chown root:root /tmp/suid_test
sudo chmod 4755 /tmp/suid_test
./suid_test # Now shows 'root'!

# This is why SUID binaries are dangerous
```

Practice Tasks:

1. Create a file only readable by owner (600)
2. Create a directory where users can write but not delete others' files (sticky bit: 1777)
3. Find all world-writable files: `find / -perm -002 -type f`
4. Audit system for dangerous permissions

■ Lab 2.2: Process Investigation and Management

Objective: Identify running processes, investigate suspicious behavior, and manage services

Duration: 45 minutes

Linux Process Investigation:

```
# View all processes
ps aux
ps -ef

# Real-time process monitoring
top
htop # Better interface, install if needed

# Find process by name
ps aux | grep ssh
pgrep -fl ssh

# View process tree
pstree -p

# Check what files a process has open
lsof -p <PID>

# Check network connections
netstat -tuln
ss -tuln # Modern replacement

# Find which process is using a port
lsof -i :22
ss -tulnp | grep :22
```

Service Management (systemd):

```
# List all services
systemctl list-units --type=service

# Check service status
systemctl status ssh

# Start/stop/restart service
sudo systemctl start ssh
sudo systemctl stop ssh
sudo systemctl restart ssh

# Enable/disable at boot
sudo systemctl enable ssh
sudo systemctl disable ssh

# View service logs
journalctl -u ssh.service
journalctl -u ssh.service --since today
```

Investigation Scenario:

Simulate suspicious process:

1. In one terminal: `nc -l 4444` (netcat listener)
2. Investigate: Find this process, identify its PID, see what port it's listening on, kill it
3. Commands: `ps aux | grep nc`, `lsof -i :4444`, `kill <PID>`

SESSION 3 LABS: Identity and Access Management

■ Lab 3.1: Implementing Multi-Factor Authentication

Objective: Set up TOTP-based MFA and understand how it works

Duration: 30 minutes

Tools: Google Authenticator app (or any TOTP app)

```
# Install library
pip install pyotp qrcode[pil]

# Create Python script: mfa_demo.py
import pyotp
import qrcode

# Generate secret
secret = pyotp.random_base32()
print(f"Secret: {secret}")

# Create provisioning URI
uri = pyotp.totp.TOTP(secret).provisioning_uri(
    name="user@example.com",
    issuer_name="MyApp"
)

# Generate QR code
qr = qrcode.make(uri)
qr.save("mfa_qr.png")
print("Scan mfa_qr.png with authenticator app")

# Verify code
totp = pyotp.TOTP(secret)
user_code = input("Enter code from app: ")
if totp.verify(user_code):
    print("✓ Authentication successful!")
else:
    print("✗ Authentication failed")
```

Understanding:

1. The secret is shared between app and server
2. TOTP generates time-based codes (30-second windows)
3. Server verifies code matches what it would generate
4. Even if password is stolen, attacker needs physical device

■ Lab 3.2: Understanding OAuth 2.0 Flow

Objective: Implement basic OAuth flow to understand authorization delegation

Duration: 45 minutes

Tools: GitHub account (for OAuth testing)

Observe OAuth in action:

1. Go to any site offering 'Login with GitHub'
2. Open browser developer tools (F12) → Network tab
3. Click 'Login with GitHub'
4. Observe the redirects:
 - Redirect to github.com/login/oauth/authorize
 - You authorize the app
 - Redirect back with authorization code in URL
 - App exchanges code for access token (server-side, you don't see this)
5. The app now has limited access to your GitHub data

Create a test OAuth app:

1. Go to GitHub Settings → Developer Settings → OAuth Apps → New OAuth App
2. Set Authorization callback URL to <http://localhost:8000/callback>
3. Note Client ID and Client Secret
4. Use a simple Flask app to test the flow (code example in GitHub OAuth docs)

SESSION 4 LABS: Cryptography in Practice

■ Lab 4.1: File Encryption with AES

Objective: Encrypt and decrypt files using industry-standard cryptography

Duration: 30 minutes

Tools: OpenSSL, Python cryptography library

Using OpenSSL (Command Line):

```
# Create test file
echo "Secret data" > plaintext.txt

# Encrypt with AES-256-CBC
openssl enc -aes-256-cbc -salt -in plaintext.txt -out encrypted.bin
# You'll be prompted for password

# View encrypted file (should be gibberish)
cat encrypted.bin

# Decrypt
openssl enc -d -aes-256-cbc -in encrypted.bin -out decrypted.txt
# Enter same password

# Verify
cat decrypted.txt
```

Python Implementation:

```
from cryptography.fernet import Fernet

# Generate key
key = Fernet.generate_key()
print(f"Key (save this!): {key}")

# Create cipher
cipher = Fernet(key)

# Encrypt
plaintext = b"Secret message"
ciphertext = cipher.encrypt(plaintext)
print(f"Encrypted: {ciphertext}")

# Decrypt
decrypted = cipher.decrypt(ciphertext)
print(f"Decrypted: {decrypted}")
```

Challenge: What happens if you try to decrypt with wrong key? Try it!

■ Lab 4.2: Password Hashing and Verification

Objective: Understand proper password storage using bcrypt

Duration: 30 minutes

```
# Install bcrypt
pip install bcrypt

import bcrypt

# Hash password
password = b"mySecretPassword123"
salt = bcrypt.gensalt(rounds=12) # Work factor
hashed = bcrypt.hashpw(password, salt)
print(f"Hash: {hashed}")
# Notice: salt is embedded in hash

# Verify password
if bcrypt.checkpw(password, hashed):
    print("✓ Password correct")
else:
    print("✗ Password incorrect")

# Try wrong password
wrong = b"wrongPassword"
if bcrypt.checkpw(wrong, hashed):
    print("✓ Password correct")
else:
    print("✗ Password incorrect")
```

Timing Test:

Run the hashing with different work factors (rounds=4, 8, 12, 14) and time each. Notice how computation time increases. This is intentional—it slows down brute-force attacks.

■ Lab 4.3: Examining TLS Certificates

Objective: Inspect SSL/TLS certificates and understand certificate chains

Duration: 30 minutes

```
# View certificate of a website
openssl s_client -connect google.com:443 -showcerts

# Press Ctrl+C after it connects

# Better: save to file and parse
echo | openssl s_client -connect google.com:443 2>/dev/null | \
openssl x509 -text -noout

# Look for:
# - Subject: CN (Common Name) = google.com
# - Issuer: Who signed it
# - Validity period: Not Before / Not After
# - Subject Alternative Names: Other domains covered
# - Public key algorithm and size

# Check expiration
echo | openssl s_client -connect google.com:443 2>/dev/null | \
openssl x509 -noout -dates
```

Certificate Analysis Tasks:

1. Check certificates for: google.com, github.com, your bank
2. Identify the root CA in each chain
3. Find a certificate about to expire (search for expired certificate check services)
4. Compare key sizes (2048-bit RSA vs 256-bit ECC)

SESSION 5 LABS: Secure Network & Infrastructure Design

■ Lab 5.1: Implementing Firewall Rules

Objective: Configure host-based firewall rules

Duration: 45 minutes

Tools: iptables (Linux) or Windows Firewall

Linux (iptables):

```
# View current rules
sudo iptables -L -n -v

# Default policy: DROP (deny all)
sudo iptables -P INPUT DROP
sudo iptables -P FORWARD DROP
sudo iptables -P OUTPUT ACCEPT

# Allow loopback
sudo iptables -A INPUT -i lo -j ACCEPT

# Allow established connections
sudo iptables -A INPUT -m state --state ESTABLISHED,RELATED -j ACCEPT

# Allow SSH (port 22)
sudo iptables -A INPUT -p tcp --dport 22 -j ACCEPT

# Allow HTTP/HTTPS
sudo iptables -A INPUT -p tcp --dport 80 -j ACCEPT
sudo iptables -A INPUT -p tcp --dport 443 -j ACCEPT

# Allow ping (ICMP)
sudo iptables -A INPUT -p icmp --icmp-type echo-request -j ACCEPT

# Log dropped packets
sudo iptables -A INPUT -j LOG --log-prefix "IPTables-Dropped: "

# Save rules (Ubuntu/Debian)
sudo iptables-save > /etc/iptables/rules.v4
```

Testing:

1. Try to connect to a blocked port: `telnet localhost 3306` (should fail)
2. SSH should work: `ssh localhost`
3. Check logs: `sudo tail -f /var/log/syslog | grep IPTables`

■ Lab 5.2: AWS VPC Design (Cloud Lab)

Objective: Design and deploy secure VPC with public/private subnets

Duration: 60 minutes

Prerequisites: AWS Free Tier account

Architecture:

- VPC: 10.0.0.0/16
- Public Subnet: 10.0.1.0/24 (web servers)
- Private Subnet: 10.0.2.0/24 (databases)
- Internet Gateway for public subnet
- NAT Gateway for private subnet outbound
- Security groups for access control

Steps:

1. Create VPC via AWS Console
2. Create subnets (public and private)
3. Create and attach Internet Gateway to VPC
4. Create NAT Gateway in public subnet
5. Configure route tables:
 - Public: 0.0.0.0/0 → Internet Gateway
 - Private: 0.0.0.0/0 → NAT Gateway
6. Create security groups:
 - Web-SG: Allow 80/443 from Internet
 - DB-SG: Allow 3306 from Web-SG only
7. Launch test EC2 instances:
 - Web server in public subnet
 - Database in private subnet
8. Verify:
 - Can reach web server from Internet
 - Cannot reach database from Internet
 - Web server CAN reach database
 - Database can reach Internet (via NAT) for updates

■ **Cost Warning:** NAT Gateways cost money (~\$0.045/hour). Remember to delete resources after lab!

Conclusion: Building Practical Expertise

Completing these labs provides hands-on experience with the foundational concepts from Sessions 1-5. Each exercise reinforces theoretical knowledge through practical application—the most effective way to prepare for technical interviews and real-world security work.

Next Steps for Continued Learning

1. **Repeat labs regularly:** Muscle memory develops through repetition
2. **Modify and experiment:** Change parameters, break things intentionally, fix them
3. **Build on basics:** Combine concepts (e.g., encrypted file storage with proper permissions)
4. **Document your learning:** Keep lab notes, create cheat sheets
5. **Join CTF competitions:** TryHackMe, HackTheBox for more advanced challenges
6. **Build a portfolio:** GitHub repos with scripts, configurations, documentation
7. **Practice explaining:** Teach concepts to others or write blog posts
8. **Stay current:** Follow security researchers, read postmortems, track CVEs

Remember: Security is a journey, not a destination. These labs are your foundation. Continue building, experimenting, and learning. Every security engineer started where you are now.